



Engenharia de Computação
Inteligência Artificial
07 - Prolog

Programação Lógica, Unificação e Busca

Organização da aula

- Retomada: LPO, fatos, regras, unificação, encadeamento reverso e retrocesso.
- Programação lógica e estrutura de um programa Prolog.
- Fatos, regras, consultas e variáveis.
- Unificação, resolução de metas, *backtracking* e recursão.
- Exemplo integrador: alcançabilidade em grafos e relação com busca em profundidade.
- Material complementar para estudo: aritmética, estruturas e listas.
- Respostas comentadas das atividades para revisão após a aula.

Fio condutor

O que muda quando fatos, regras e mecanismos de inferência deixam de ser apenas teoria lógica e passam a constituir um programa executável?

O que é Prolog?

- Prolog é uma linguagem de **programação lógica**.
- Seu núcleo trabalha com fatos e regras próximas de **cláusulas de Horn**, com raízes na Lógica de Primeira Ordem.
- O núcleo da linguagem possui padronização ISO; implementações concretas adicionam extensões.
- Nesta disciplina utilizaremos a sintaxe e o comportamento do **SWI-Prolog**.

Discussão

Em uma linguagem imperativa, normalmente descrevemos *como* calcular. Em Prolog, procuramos descrever *o que é verdadeiro* sobre o domínio e quais relações devem valer.

Programação declarativa e leitura operacional

Leitura declarativa

O programa descreve relações entre objetos e condições para que essas relações sejam verdadeiras.

Leitura operacional

O Prolog tenta satisfazer metas escolhendo cláusulas, unificando termos e executando submetas em uma ordem definida.

Observação

Entender Prolog exige as duas leituras. Um programa pode estar logicamente correto e ainda assim ter comportamento operacional ruim, repetir respostas ou não terminar.

Termos em Prolog

Os principais tipos de termos usados nesta aula são:

- **átomos:** `ana`, `azul`, `no_centro`;
- **números:** `3`, `4.5`;
- **variáveis:** `X`, `Pessoa`, `Resultado`;
- **termos compostos:** `ponto(2,4)`, `progenitor(ana,bruno)`.

Observação

Átomos normalmente começam com letra minúscula; variáveis começam com letra maiúscula ou com sublinhado.

Consultas: perguntar à base de conhecimento

Com o programa carregado, podemos consultar:

```
?- progenitor(ana, bruno).  
true.  
  
?- progenitor(ana, diego).  
false.
```

Pergunta

O segundo resultado significa que é logicamente impossível Ana ser progenitora de Diego, ou apenas que o programa não consegue provar isso com o conhecimento disponível?

Consultas com variáveis

Uma variável transforma a consulta em uma busca por valores que a tornam verdadeira.

```
?- progenitor(bruno, X).  
X = clara ;  
X = diego ;  
false.
```

- a primeira resposta fornece uma substituição para X;
- o ponto e vírgula solicita outra solução;
- ao esgotar as alternativas, não há nova solução a retornar.

Conexão com LPO

A resposta `X = clara` é uma substituição encontrada por unificação durante a busca pela prova.

Exercício 1 - resposta comentada

Respostas esperadas

Cada consulta é resolvida procurando fatos que unifiquem com a meta.

```
?- progenitor(bruno, clara).
true.
```

```
?- progenitor(ana, X).
X = bruno.
```

```
?- progenitor(X, eva).
X = clara.
```

```
?- progenitor(bruno, X).
X = clara ;
X = diego ;
false.
```

Onde aparece o retrocesso?

Unificação: o mecanismo de encaixe

Retomando

Na Aula 06, unificar significava encontrar uma substituição que tornasse dois termos idênticos.

Exemplos:

$\text{progenitor}(X, \text{bruno})$ e $\text{progenitor}(\text{ana}, \text{bruno})$

unificam com

$\{X/\text{ana}\}$.

Já

$\text{progenitor}(X, \text{bruno})$ e $\text{progenitor}(\text{ana}, \text{clara})$

não unificam, pois os segundos argumentos são incompatíveis.

Unificação diretamente no Prolog

O operador = tenta unificar dois termos:

```
?- X = ana.
```

```
X = ana.
```

```
?- pessoa(X, 20) = pessoa(carla, Idade).
```

```
X = carla,
```

```
Idade = 20.
```

```
?- ponto(X, 2) = ponto(3, 4).
```

```
false.
```

Observação

= não significa “atribuição” como em muitas linguagens imperativas. Seu papel é unificação.

Exercício 2 - escrevendo regras

Em duplas - 7 minutos

Usando somente os fatos `progenitor/2`, escreva regras para as relações abaixo.

- 1 `filho(Filho, Progenitor)` - relação inversa de `progenitor/2`;
- 2 `irmaos(X, Y)` - X e Y possuem um progenitor em comum;
- 3 `neto(Neto, Avo)` - existe uma pessoa intermediária entre avô e neto.

Teste mental

A regra de irmãos também permite concluir `irmaos(clara, clara)`? Se sim, como você impediria essa resposta?

Exercício 2 - resposta comentada

1. Relação inversa: filho/2

```
filho(Filho, Progenitor) :-  
    progenitor(Progenitor, Filho).
```

2. Irmãos: um progenitor em comum

```
irmaos(X, Y) :-  
    progenitor(P, X),  
    progenitor(P, Y),  
    X \= Y.
```

3. Neto: uma pessoa intermediária

```
neto(Neto, Avo) :-  
    progenitor(Avo, P),  
    progenitor(P, Neto).
```

Por que precisamos de $X \neq Y$?

Sem a restrição, ter um progenitor em comum é suficiente:

```
irmaos(X, Y) :- progenitor(P, X), progenitor(P, Y).
```

Considere a consulta:

```
?- irmaos(clara, clara).
```

Se a KB contém `progenitor(bruno, clara)`., as duas submetas tornam-se exatamente o mesmo fato:

```
progenitor(bruno, clara), progenitor(bruno, clara).
```

Consequência

A consulta teria sucesso. Para excluir esse caso, declaramos explicitamente $X \neq Y$. Prolog executa o que foi formalizado; ele não acrescenta a interpretação intuitiva de “irmãos”.

Da intenção à regra formal

A primeira versão expressava apenas:

$$\exists P \ (Progenitor(P, X) \wedge Progenitor(P, Y)).$$

Isso é verdadeiro inclusive quando $X = Y$.

Para representar a relação desejada, acrescentamos a condição:

$$X \neq Y.$$

Assim, a ideia completa fica:

$$Irmaos(X, Y) \leftarrow Progenitor(P, X) \wedge Progenitor(P, Y) \wedge X \neq Y.$$

Mensagem central

Prolog não completa o significado que pretendemos. Ele executa exatamente as condições que formalizamos na regra.

Prova direta e prova por regras

Nesta aula, usaremos dois nomes didáticos para acompanhar a execução:

Prova direta por fato

A meta unifica diretamente com um fato da base de conhecimento.

Prova por regra

A meta unifica com a cabeça de uma regra e é substituída pelas submetas de seu corpo.

Não confundir

“Direta” aqui significa **resolver a meta com um fato**. O mecanismo de execução do Prolog continua sendo orientado pela meta, isto é, essencialmente encadeamento reverso — não encadeamento direto.

Prova por regra - passo 1: escolher e unificar a regra

Meta inicial:

avo ou avoa(bruno, eva)

A cabeça da regra é:

$$avo_ou_avoa(X, Z)$$

A unificação produz:

 $\{X/bruno, Z/eva\}.$

Aplicando essa substituição ao corpo, a meta original é substituída por:

$$\text{progenitor}(\text{bruno}, Y), \quad \text{progenitor}(Y, \text{eva}).$$

Observação

A consulta ainda não foi provada. A regra apenas transformou uma meta em duas submetas.

Prova por regra - passo 2: resolver a primeira submeta

Primeira submeta, pois o Prolog trabalha da esquerda para a direita:

progenitor(bruno, Y)

O primeiro fato compatível é:

```
progenitor(bruno, clara).
```

Logo:

 $\{Y/ clara\}.$

A segunda submeta passa a ser:

```
progenitor(clara, eva).
```

Ponto de escolha

Como também existe progenitor (bruno, diego), fica registrado um ponto de escolha para Y .

Prova por regra - passo 3: resolver a segunda submeta

Agora é necessário provar:

progenitor(clara, eva).

Esse fato existe diretamente na base de conhecimento. Portanto:

progenitor(*clara, eva*) ✓

e todas as submetas do corpo foram satisfeitas.

Conclusão

A consulta `avo_ou_avoa(bruno, eva)` tem sucesso porque foi construída uma prova completa a partir da regra e dos fatos.

Registro da prova por regra

Etapa	Meta escolhida	Resultado
1	avo_ou_avoa (bruno, eva)	regra selecionada; X=bruno, Z=eva
2	progenitor (bruno, Y)	fato selecionado; Y=clara
3	progenitor (clara, eva)	fato encontrado; sucesso

Retomando

A execução pode ser acompanhada como uma sequência repetida de **meta** → **cláusula** → **unificação** → **novas metas**.

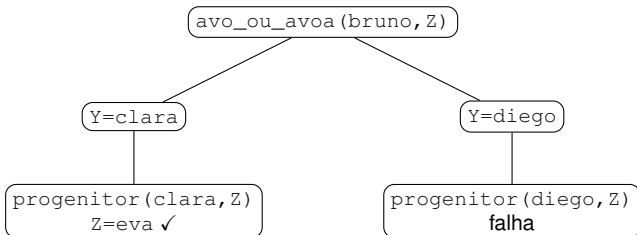
E quando pedimos outra solução?

Considere agora uma consulta que deixa a resposta aberta:

```
?- avo_ou_avoa(bruno, Z) .
```

- 1 primeira escolha: `Y = clara`;
- 2 a submeta `progenitor(clara, Z)` produz `Z = eva`;
- 3 ao digitar `;`, o Prolog volta ao ponto de escolha de `progenitor(bruno, Y)`;
- 4 desfaz `Y = clara` e tenta `Y = diego`;
- 5 passa a procurar `progenitor(diego, Z)`;
- 6 não há fato compatível: esse ramo falha e não surgem novas respostas.

Árvore de prova e pontos de escolha



Observação

O ramo da esquerda produz uma resposta. O ramo da direita só é explorado quando se pede outra solução ou quando a primeira alternativa não permite concluir a consulta.

Exercício 4 - e se pedirmos outra resposta?

A consulta é totalmente instanciada:

?- ancestral(ana, eva).

Ela pergunta apenas se existe uma prova. Na base atual há uma única cadeia que demonstra essa relação.

- ao solicitar outra solução com `;`, o Prolog volta aos pontos de escolha anteriores;
- tenta, por exemplo, a alternativa `progenitor(bruno,diego)`;
- como `diego` não leva a `eva`, essa alternativa falha;
- não restando outra prova, a busca termina.

Observação

Consultas com variáveis podem produzir novas substituições. Nesta consulta fechada, uma prova alternativa apenas confirmaria novamente o mesmo resultado lógico: sucesso.

Falha não é o mesmo que falsidade clássica

- quando uma consulta falha, o Prolog informa que **não encontrou uma prova** com o programa disponível;
- isso não é automaticamente equivalente a demonstrar a negação lógica da consulta;
- muitos programas Prolog adotam uma visão de mundo fechado: o que não é provado é tratado operacionalmente como falha.

Cuidado conceitual

Programação lógica e lógica clássica estão relacionadas, mas o modelo operacional do Prolog introduz escolhas e hipóteses que precisam ser compreendidas separadamente.

Variáveis e variável anônima

```
tem_filho(X) :- progenitor(X, _).
```

- `x` interessa porque queremos saber quem tem filho;
- `_` representa uma variável cujo valor não precisamos recuperar;
- cada ocorrência isolada de `_` deve ser entendida como uma nova variável anônima.

Interpretação

O que a regra afirma sobre o nome do filho? Nada: apenas exige que algum segundo argumento exista em um fato compatível.

```
% Conjuncao: ambas as metas devem ter sucesso
mae(X, Y) :- progenitor(X, Y), mulher(X).

% Disjuncao: uma das alternativas deve ter sucesso
responsavel(X, Y) :- mae(X, Y) ; pai(X, Y).
```

- 45 / 83

Atividade curta - respostas esperadas

Com a ordem dos fatos e regras usada na aula:

```
?- progenitor(bruno, X).
X = clara ;
X = diego ;
false.
```

```
?- avo_ou_avoa(X, eva).
X = bruno ;
false.
```

```
?- ancestral(ana, X).
X = bruno ;
X = clara ;
X = diego ;
X = eva ;
false.
```

```
?- ancestral(diego, X).
false.
```

Atividade curta - onde aparecem os pontos de escolha?

- `progenitor(bruno, X)` possui duas correspondências diretas: clara e diego.
- `avo_ou_avoa(X, eva)` encontra apenas uma resposta final, mas a busca testa alternativas internas até construir a prova.
- `ancestral(ana, X)` combina duas cláusulas para `ancestral/2` e vários fatos `progenitor/2`; o retrocesso é essencial para enumerar todos os descendentes.
- `ancestral(diego, X)` termina em falha após esgotar as cláusulas e fatos compatíveis.

Leitura operacional

Um ponto de escolha apenas registra uma alternativa ainda disponível. Ele não garante que essa alternativa produzirá outra resposta.

SWI-Prolog e SWISH

Para os exemplos da disciplina, duas opções são suficientes:

- **SWI-Prolog**: implementação completa para execução local;
- **SWISH**: ambiente web baseado em SWI-Prolog para editar e executar programas no navegador.

Observação

O foco da aula não é configuração de IDE ou integração com C++/Java. O objetivo é compreender o modelo lógico e operacional da linguagem.

Erros conceituais frequentes

- interpretar = como atribuição em vez de unificação;
- esquecer que variáveis começam com letra maiúscula ou sublinhado;
- interpretar falha de prova como negação lógica completa;
- ignorar que a ordem de metas e cláusulas afeta a execução;
- criar recursão sem condição de parada efetiva;
- esquecer que uma nova resposta pode exigir retrocesso até um ponto de escolha anterior.

Síntese do núcleo da aula

- Prolog organiza programas em fatos, regras e consultas;
- unificação transforma metas genéricas em casos concretos;
- a execução tradicional combina encadeamento reverso, busca em profundidade e retrocesso;
- pontos de escolha preservam alternativas que podem ser retomadas após falha ou solicitação de outra resposta;
- recursão permite expressar relações de tamanho arbitrário;
- o exemplo de grafos mostra que uma consulta pode ser entendida como busca por uma prova.

Cheque sua compreensão

Qual é a diferença entre descrever uma relação em Prolog e escrever explicitamente um algoritmo imperativo para percorrê-la?

Até aqui: núcleo da Aula 07

Fatos $\rightarrow$ regras $\rightarrow$ consultas $\rightarrow$ unificação $\rightarrow$
prova $\rightarrow$ retrocesso $\rightarrow$ recursão $\rightarrow$ busca

Ponto de parada para o encontro de 100 minutos

Até este ponto estão os mecanismos centrais de Prolog para a disciplina. O restante do material foi mantido no PDF como referência para estudo e apoio à Lista 05.

Antes de avançar

Você consegue explicar, sem executar o programa, como uma consulta gera submetas, substituições, pontos de escolha e retrocessos?

Estudo complementar - exemplos de testes

Testar um predicado significa verificar casos de sucesso, falha e, quando pertinente, geração de respostas.

```
?- dobro(4, Y).                % Y = 8
?- primeiro(X, [a,b,c]).        % X = a
?- ultimo(X, [a,b,c]).          % X = c
?- tamanho([a,b,c], N).         % N = 3
?- concatena([a,b], [c,d], L).  % L = [a,b,c,d]
?- descendente(eva, ana).       % true
?- alcanca(a, h).               % true
?- alcanca(h, a).               % false, se nao houver ciclo
```

Boa prática

Para cada predicado, procure pelo menos um teste que deva ter sucesso e um que deva falhar. Quando houver variáveis de saída, confira também as substituições produzidas.

Desafio - uma solução possível

Podemos manter uma lista de visitados e construir o caminho em ordem inversa durante a busca:

```

caminho(Origem, Destino, Caminho) :-
    caminho_aux(Origem, Destino, [Origem], Rev),
    reverse(Rev, Caminho).

caminho_aux(Destino, Destino, Visitados, Visitados).

caminho_aux(Atual, Destino, Visitados, Caminho) :-
    aresta(Atual, Proximo),
    \+ member(Proximo, Visitados),
    caminho_aux(Proximo, Destino,
                [Proximo|Visitados], Caminho).

```

Papel de Visitados

A lista impede que um vértice seja expandido novamente no mesmo caminho. Ela exerce o mesmo papel conceitual do controle de estados repetidos nos algoritmos de busca.

Desafio - relação com DFS

- `aresta (Atual, Proximo)` gera sucessores na ordem em que os fatos aparecem na base;
- o Prolog escolhe o primeiro sucessor disponível e chama recursivamente `caminho_aux/4`;
- portanto, aprofunda um ramo antes de testar os demais;
- quando uma alternativa falha, o retrocesso retorna ao ponto de escolha anterior e tenta outro sucessor;
- esse comportamento corresponde à lógica de exploração em profundidade da DFS.

Resposta à reflexão

O problema continua sendo encontrar um caminho. A estratégia de exploração continua semelhante à DFS. O que muda é a forma de expressá-la: em Prolog, grande parte do controle de busca aparece implicitamente no mecanismo de unificação, recursão e retrocesso da linguagem.

Referências e ambiente de execução

- SWI-Prolog Reference Manual e documentação oficial.
- ISO/IEC 13211-1 - núcleo da linguagem Prolog.
- Bratko, I. *Prolog Programming for Artificial Intelligence*.
- Clocksin, W. F.; Mellish, C. S. *Programming in Prolog*.
- Material-base anterior da disciplina, revisado e reorganizado para a sequência atual.

Próxima aula: Lista 05

Como aplicar fatos, regras, unificação, recursão e retrocesso na construção e análise de programas Prolog?

Lista 05 - prática em Prolog

A próxima aula será destinada à resolução da Lista 05, consolidando o núcleo estudado nesta aula e exigindo consulta ao material complementar quando necessário.

Dúvidas



Dúvida é o começo da sabedoria.